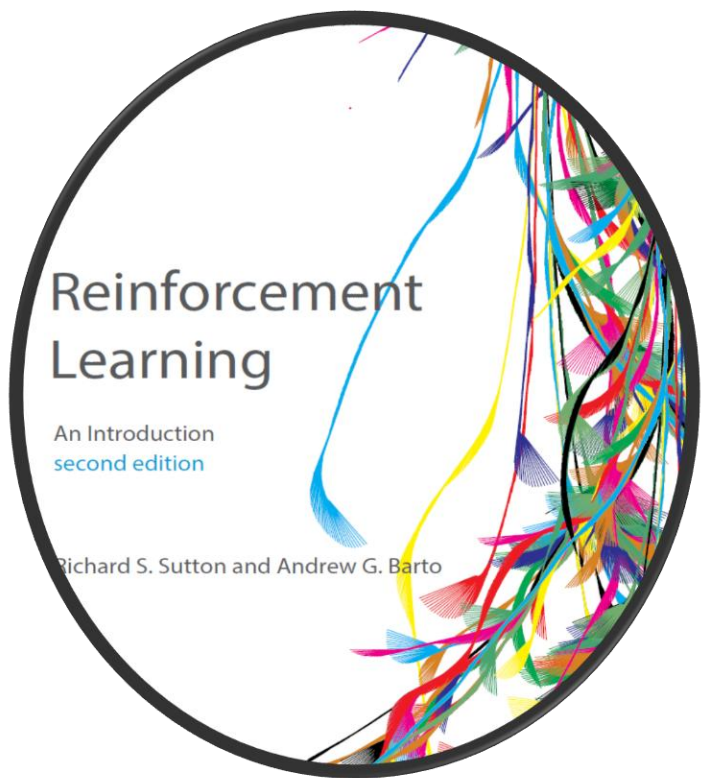




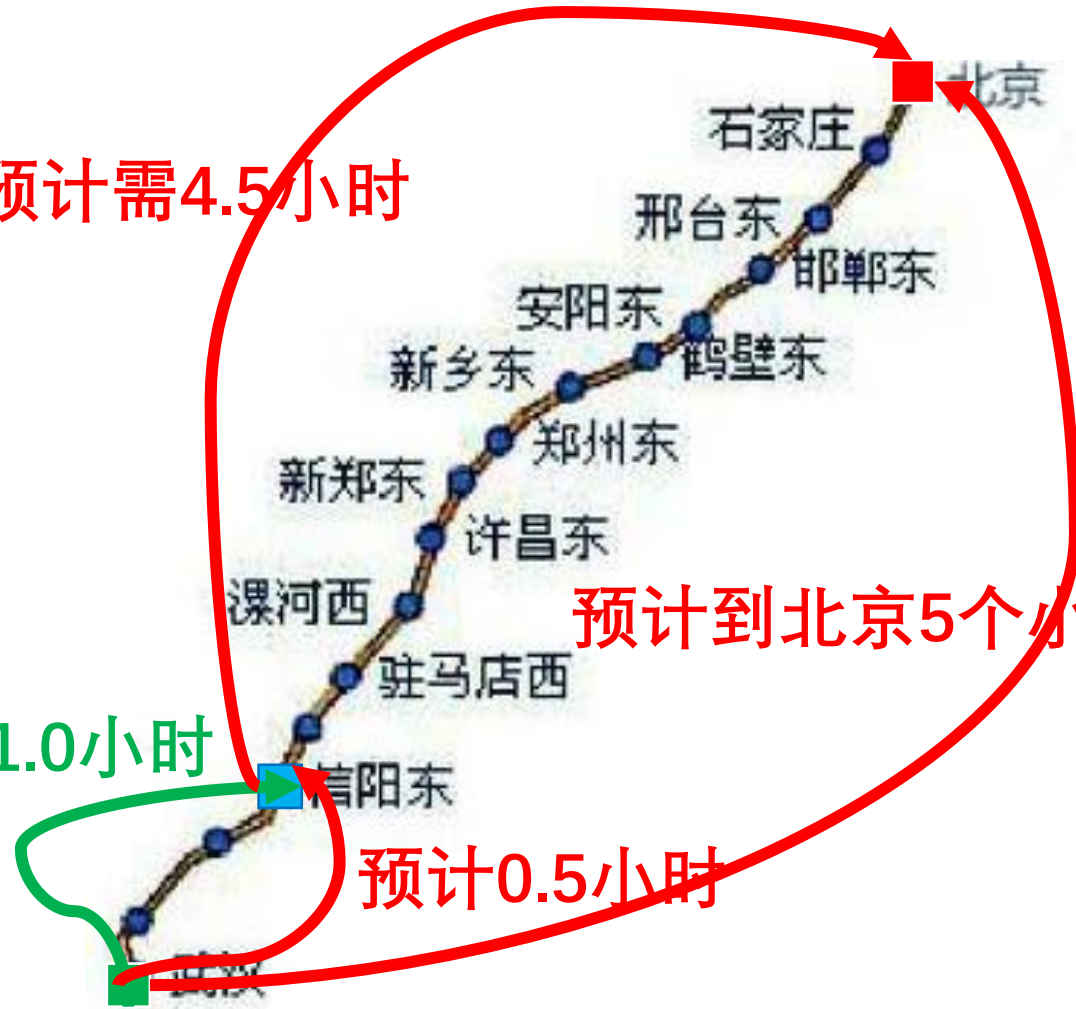
第六章 时序差分学习 (Temporal-Difference Learning)

何新卫

华中农业大学信息学院



预计需4.5小时



预计到北京5个小时

预计0.5小时

实际花费1.0小时



回顾

□ 估计价值

- 估计**状态价值** $v_{\pi}(s)$, for all $s \in \mathcal{S}$
- 估计**状态-动作价值** $q_{\pi}(s, a)$, for all $s \in \mathcal{S}, a \in \mathcal{A}$

□ 方法:

- 动态规划: 已知确定环境
- 蒙特卡洛方法: 已知或未知环境, 使用经验数据估算价值
- **时序差分方法: 融合了动态规划和蒙特卡洛思想 (本章重点)**

回顾：广义策略迭代

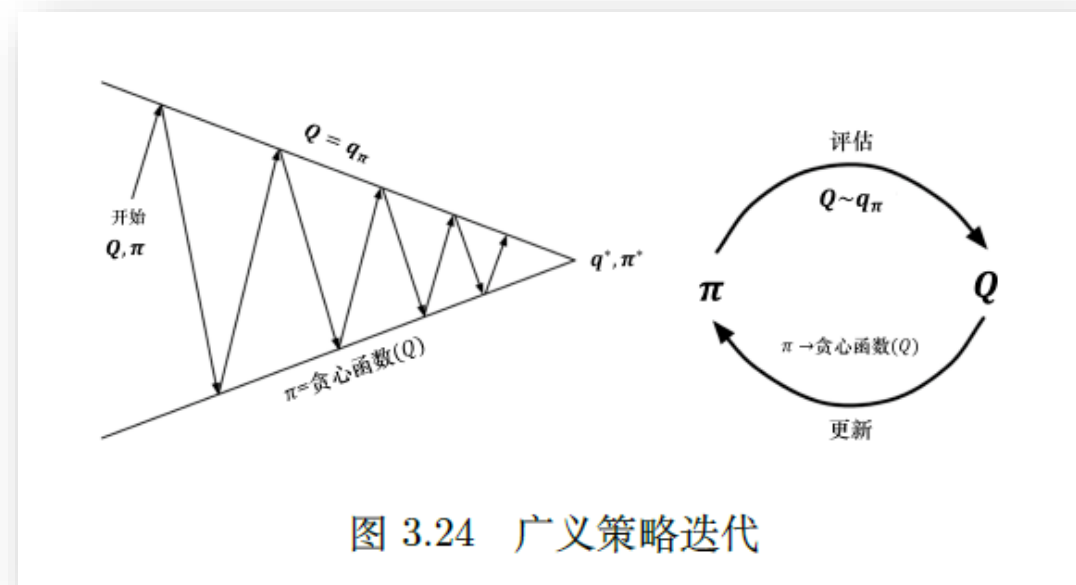
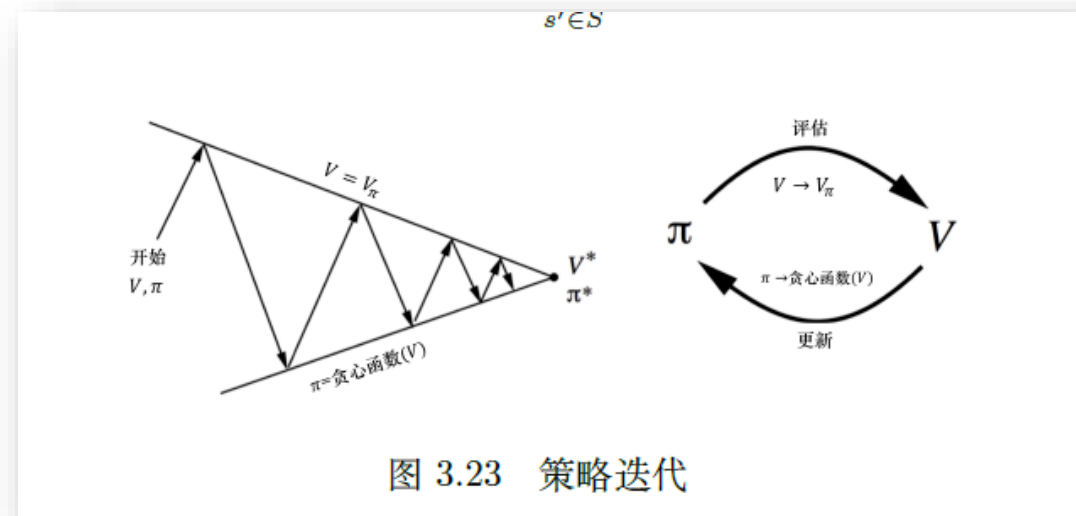
□兼容蒙特卡洛和时序差分

□策略评估 (**Policy Evaluation**)

➤ 给定任意 π ，计算 π 对应状态价值函数 $v_{\pi}(\cdot)$

□策略提升 (**Policy Improvement**)

➤ 基于价值函数，进行贪心策略改进



策略评估（或预测）

回顾

□ 动态规划，利用自举思想（即基于旧估值计算出新估值）更新

$$V_{k+1}(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V_k(s')] \text{ for all } s \in \mathcal{S}$$

□ MC方法: 智能体遵循策略 π 和环境交互获取数据。 MC到最终状态，

计算状态回报 $V(S_t)$ 。 every-visit MC的计算如下 ($\alpha = \frac{1}{N(S_t)}$) :

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$



TD预测 (TD Prediction)

□和动态规划一样，用到**自举思想**，和MC一样用到经验数据，但MC用到整个经验数据，**而TD只需要一步转移的数据**，因此更新频率更快

□TD方法，仅需要下个时刻奖励 R_{t+1} ，即可估算更新

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

时序差分更新目标 (target)

思考：为什么叫做目标值？

□上述方法被称为TD(0)，或者one-step TD

□更一般的，有TD(λ) 和n-step TD方法

TD(0)伪代码

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

Loop for each step of episode:

$A \leftarrow$ action given by π for S

Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal

□TD(0)方法的更新基于已存在的估值，我们也可以说这里用到了自举的思想（bootstrap）（类似DP方法）。

TD(0), DP, 和Monte Carlo

对于所有 $s \in \mathcal{S}$:

$$v_{\pi}(s) \triangleq \underline{\mathbb{E}_{\pi} [G_t | S_t = s]}$$

$$= \mathbb{E}_{\pi} [R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$= \underline{\mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s]}$$

MC方法 采样轨迹估计价值

DP方法，基于对环境的完整知晓，
无需采样轨迹，直接利用 $V(S_{t+1})$
估计未知 $v_{\pi}(S_{t+1})$

□ TD(0)通过**一步采样**估计价值，与此同时，利用自举方式提升 V

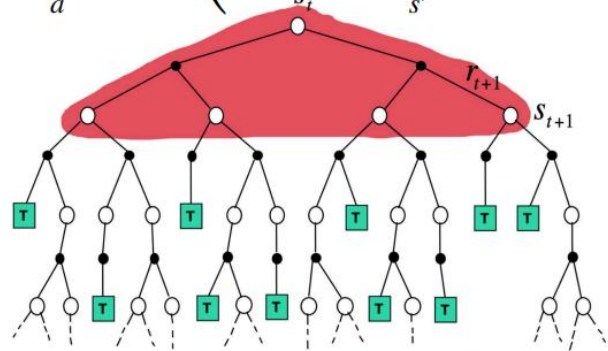
TD(0), DP, 和Monte Carlo

□价值估计分类

- 有模型方法 (Model-based) : DP, 不依赖智能体和环境交互数据
- 免模型方法 (Model-Free): TD, MC, 依赖于智能体和环境交互数据

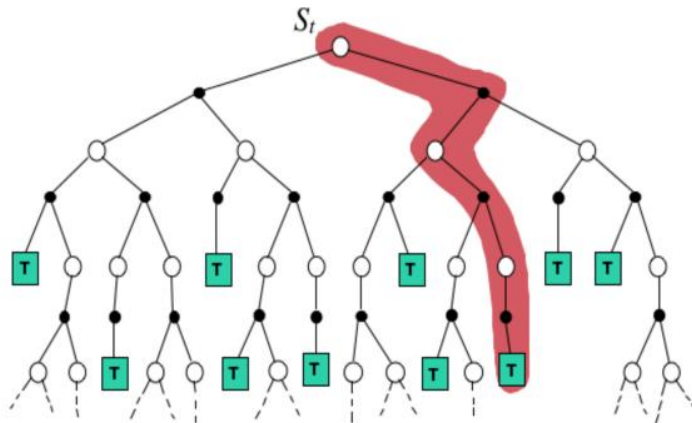
□备份图

$$V(s) \leftarrow \sum_a \pi(a|s) \left(r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right)$$



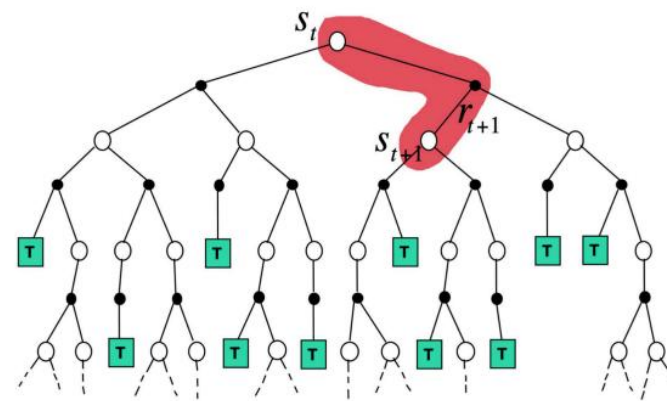
DP

$$V(s) \leftarrow V(s) + \alpha(G_t - V(s))$$



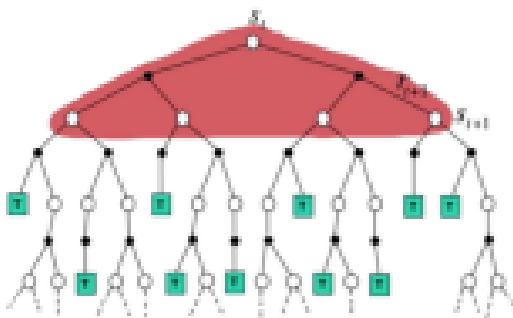
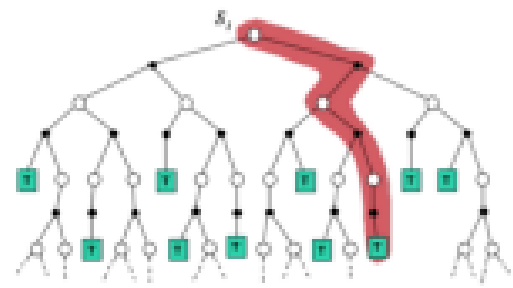
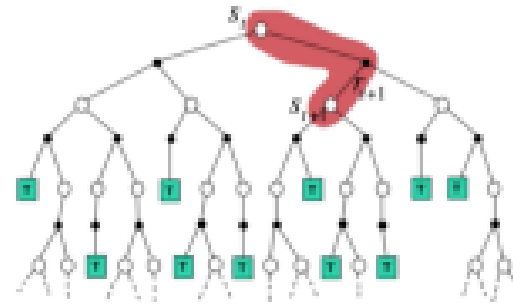
MC

$$V(s) \leftarrow V(s) + \alpha(r' + \gamma V(s') - V(s))$$



TD(0)

总结

	Diagram	Bootstrapping? (update involves an estimate)	Sampling?
DP (Dynamic Programming)		Yes	No
MC (Monte Carlo)		No	Yes
TD (Temporal Difference) TD(0)		Yes	Yes

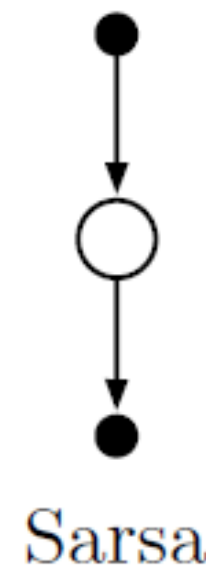
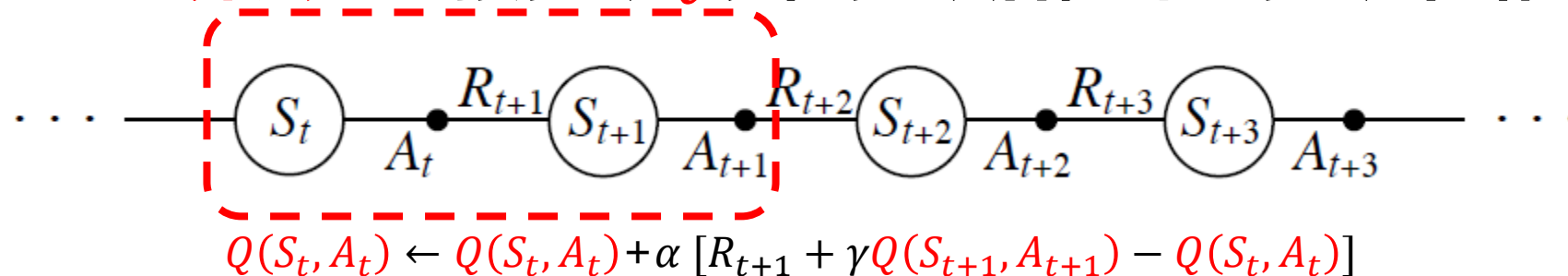
策略提升（或控制）

On-Policy TD Control

□ **基于预测的价值函数，可进一步对策略进行改进**

➤ **未知环境中**，需估计状态-动作价值函数 $q_{\pi}(s, a)$ ，for all $s \in \mathcal{S}, a \in \mathcal{A}$ ，
(注：估计状态价值函数 $v_{\pi}(s)$ 没有意义)

□ **Sarsa算法：直接更新 Q 表，更新后，即可更新策略**



➤ **基于五元组 $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ 完成更新**

➤ **首字母是Sarsa的命名由来**

Sarsa算法

□ 基于 Sarsa 设计同策略 (on-policy) 控制算法。

- 估计行为策略 π 的动作价值函数 q_π ，同时基于 q_π 朝着贪婪方向改进 π 。
- Sarsa 算法中 π 可以使用 “ ϵ -贪婪” 策略。
- 只要所有状态-动作对被无限次访问，则Sarsa以概率 1 达到最优策略和动作-价值函数
- 通过设置 $\epsilon = 1/t$ ，策略收敛到贪婪策略

Sarsa算法

Sarsa（同策略的时序差分控制）用来估计 $Q \approx Q^*$

算法参数：步长大小 $\alpha \in (0,1]$ ，一个极小值 $\epsilon > 0$ 。

对于所有的 $s \in S^+, a \in A(s)$ ，随机初始化 $Q(s, a)$ ，除非 $Q(\text{终点}, \cdot) = 0$ 。

对于每一个回合进行循环。

初始化 S 。

使用从 Q 中衍生出的策略（例如 ϵ -贪心策略）从 S 中选择 A 。

对一个回合中的每一步进行循环。

执行动作 A ，观测 R, S' 。

使用从 Q 中衍生出的策略（例如 ϵ -贪心策略）从 S' 中选择 A' 。

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$ 。

$S \leftarrow S'; A \leftarrow A'$ 。

直到 S 到达终点。

目标值

当前值

控制
预测问题

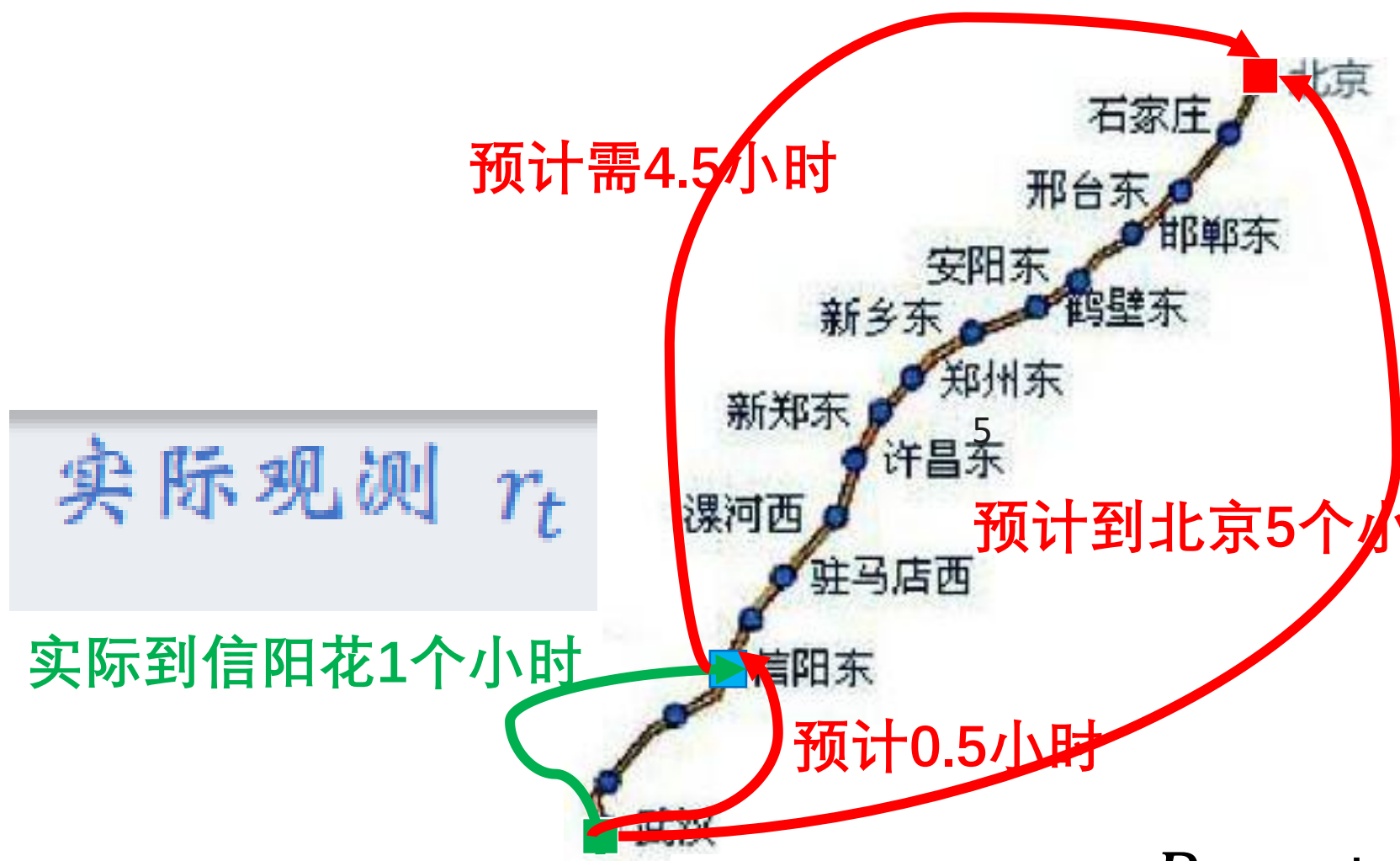
$$\begin{aligned} n = 1(\text{Sarsa}) \quad Q_t^1 &= r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) \\ n = 2 \quad Q_t^2 &= r_{t+1} + \gamma r_{t+2} + \gamma^2 Q(s_{t+2}, a_{t+2}) \\ &\vdots \\ n = \infty(\text{MC}) \quad Q_t^\infty &= r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-t-1} r_T \end{aligned}$$

目标值

不断用下一步的Q值更新当前步的Q值

为什么Sarsa是同策略 (on-policy) ?
它优化的目标策略是它实际执行的行为策略，它直接
用下一步去执行的动作去优化Q 表格。

理解时序差分：坐火车去北京举例



$$\begin{array}{r} 1+4.5 \\ \hline \text{target} \end{array} \quad \begin{array}{r} 5 \\ \hline \text{old} \end{array}$$
$$5 + 2(1+4.5-5)$$
$$= 5 + 2 \cdot 0.5$$

$$R_{t+1} + V(S_{t+1}) \longleftrightarrow V(S_t)$$

Q学习算法: Off-policy TD Control

□ 最优的动作价值函数和当前遵循的策略无关

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Q学习（异策略的时序差分控制）用来估计 $\pi \approx \pi^*$

算法参数：步长大小 $\alpha \in (0, 1]$ ，一个极小值 $\epsilon > 0$ 。

对于所有的 $s \in S^+$, $a \in A(s)$ ，随机初始化 $Q(s, a)$ ，除非 $Q(\text{终点}, \cdot) = 0$ 。

对每一个回合进行循环。

初始化 S 。

对一个回合中的每一步进行循环。

使用从 Q 中衍生出的策略（例如 ϵ -贪心策略）从 S 中选择 A 。

执行动作 A ，观测 R, S' 。

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right]$$

$S \leftarrow S'$ 。

直到 S 到达终点。

$$a_t = \begin{cases} \operatorname{argmax}_a Q(s_t, a; w), & \text{以概率 } (1 - \epsilon) \\ \text{均匀抽取 } \mathcal{A} \text{ 中的一个动作,} & \text{以概率 } \epsilon. \end{cases}$$

行为策略: ϵ -贪婪策略

目标策略: 更新贪心策略下的 Q 表

$$\pi(s_{t+1}) = \operatorname{argmax}_{a'} Q(s_{t+1}, a')$$

□ Q学习更早被提出，SARSA是Q学习的改进版本

Sarsa（同策略的时序差分控制）用来估计 $Q \approx Q^*$

算法参数：步长大小 $\alpha \in (0,1]$ ，一个极小值 $\epsilon > 0$ 。

对于所有的 $s \in S^+$, $a \in A(s)$ ，随机初始化 $Q(s,a)$ ，除非 $Q(\text{终点}, \cdot) = 0$ 。

对于每一个回合进行循环。

初始化 S 。

使用从 Q 中衍生出的策略（例如 ϵ -贪心策略）从 S 中选择 A 。

对一个回合中的每一步进行循环。

执行动作 A ，观测 R, S' 。

使用从 Q 中衍生出的策略（例如 ϵ -贪心策略）从 S' 中选择 A' 。

$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)]$ 。

$S \leftarrow S'; A \leftarrow A'$ 。

直到 S 到达终点。

(a) Sarsa

Q学习（异策略的时序差分控制）用来估计 $\pi \approx \pi^*$

算法参数：步长大小 $\alpha \in (0,1]$ ，一个极小值 $\epsilon > 0$ 。

对于所有的 $s \in S^+$, $a \in A(s)$ ，随机初始化 $Q(s,a)$ ，除非 $Q(\text{终点}, \cdot) = 0$ 。

对每一个回合进行循环。

初始化 S 。

对一个回合中的每一步进行循环。

使用从 Q 中衍生出的策略（例如 ϵ -贪心策略）从 S 中选择 A 。

执行动作 A ，观测 R, S' 。

$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \max_a Q(S',a) - Q(S,A)]$

$S \leftarrow S'$ 。

直到 S 到达终点。

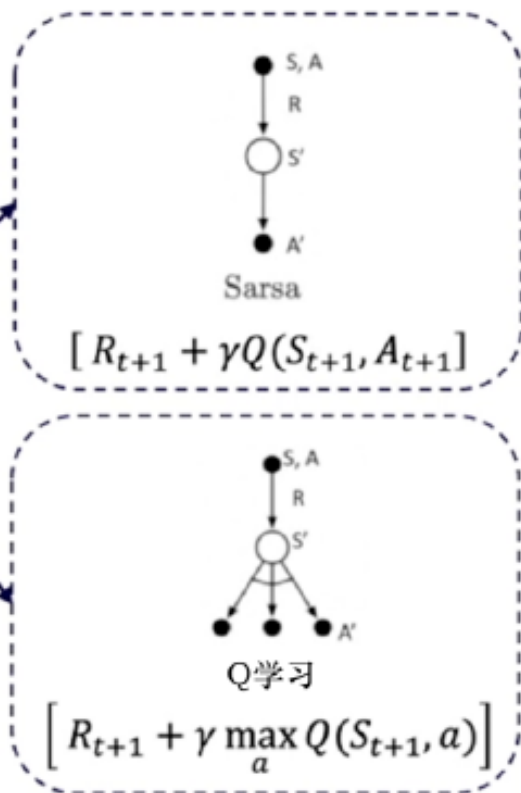
(b) Q 学习

图 3.33 Sarsa 与 Q 学习的伪代码

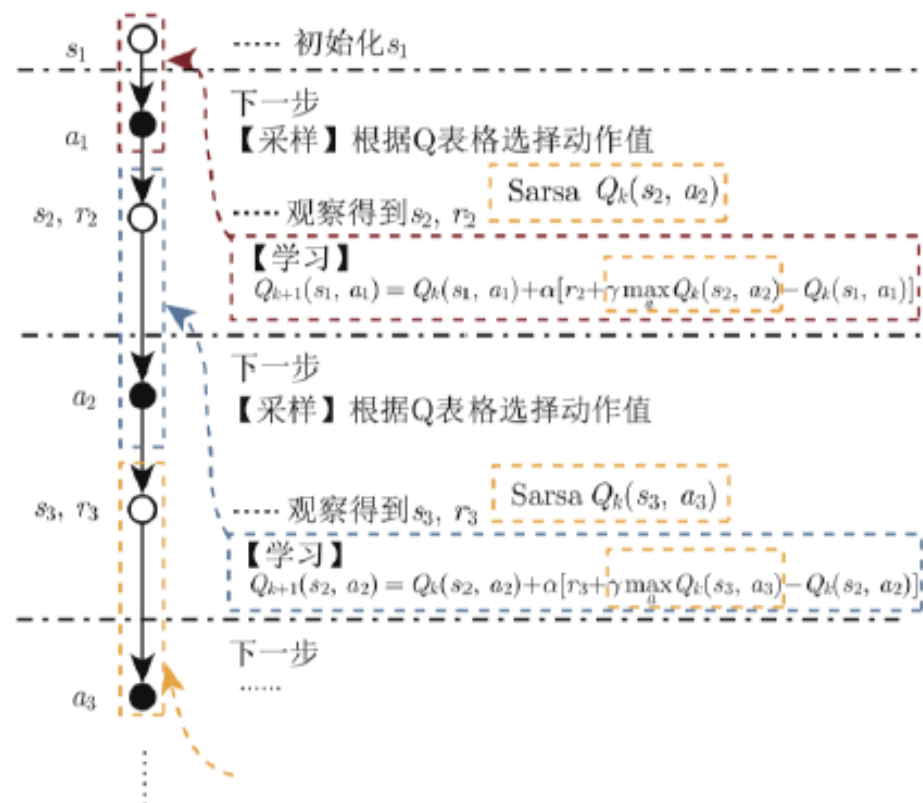
如图 3.34a 所示，Sarsa 用自己的策略产生了 S, A, R, S', A' 这条轨迹，然后用 $Q(s_{t+1}, a_{t+1})$ 去更新原本的 Q 值 $Q(s_t, a_t)$ 。但是 Q 学习并不需要知道我们实际上选择哪一个动作，它默认下一个动作就是 Q 值最大的那个动作。Q 学习知道实际上行为策略可能会有 0.1 的概率选择别的动作，但 Q 学习并不担心受到探索的影响，它默认按照最佳的策略去优化目标策略，所以它可以更大胆地去寻找最优的路径，它表现得比 Sarsa 大胆得多。

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [Q \text{ 目标值} - Q(S_t, A_t)]$$

目标



(a) Sarsa 与 Q 学习的目标



(b) Q 学习的流程

图 3.34 Sarsa 与 Q 学习的区别

Q学习算法: 为什么是Off-policy?

- 行为策略 (behavior policy): 随机
- 目标策略 (target policy): 贪心进行价值更新 (对应目标策略为贪心策略)



- 对于 Sarsa，它的更新公式必须使用来自当前策略采样得到的五元组 (s, a, r, s', a') ，因此它是在线策略学习方法；
- 对于 Q-learning，它的更新公式使用的是四元组 (s, a, r, s') 来更新当前状态动作对的价值 $Q(s, a)$ ，数据中的 s 和 a 是给定的条件， r 和 s' 皆由环境采样得到，该四元组并不需要一定是当前策略采样得到的数据，也可以来自行为策略，因此它是离线策略算法。

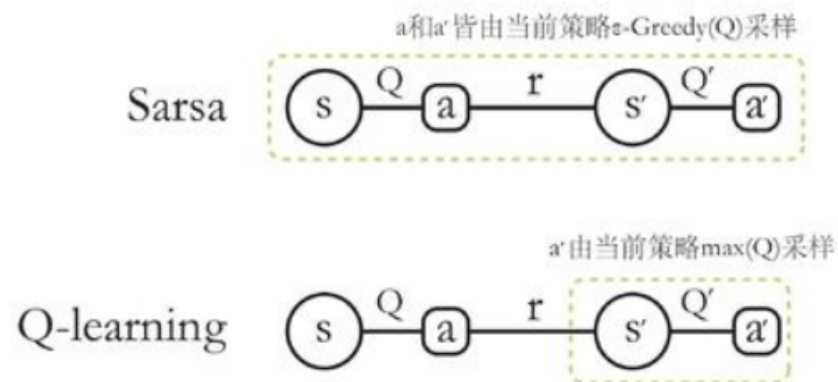


图5-1 Sarsa 和 Q-learning 的对比

异策略的好处是可以用行为策略收集经验，把 (s_t, a_t, r_t, s_{t+1}) 这样的四元组记录到一个数组里，在事后反复利用这些经验去更新目标策略。这个数组被称作经验回放数组 (replay buffer)，这种训练方式被称作经验回放 (experience replay)。注意，经验回放只适用于异策略，不适用于同策略，其原因是收集经验时用的行为策略不同于想要训练出的目标策略。

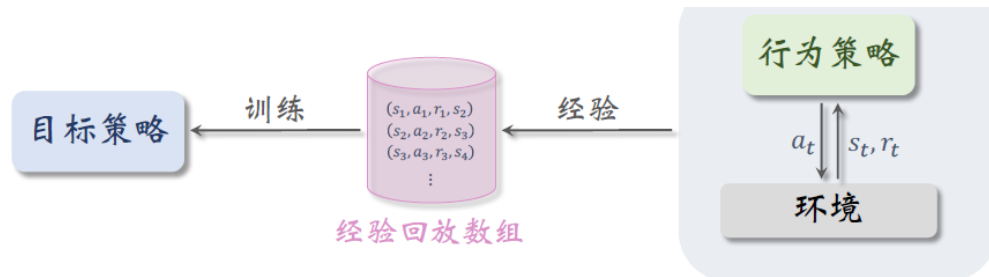


图 4.7: 异策略。

Q学习使用__控制智能体和环境进行交互。

- ☒ A 行为策略 (behavior policy)
- ☐ B 目标策略 (target policy)

提交

SARSA是___。

A

同策略时序差分控制算法

B

异策略时序差分控制算法

提交